MCSP-232

CampusConnect

A Role-Based Academic Social Network

By

Sidharth Malik

MCA Final Year Student

Under the Guidance Of

Dr. Avaneesh Anand Singh

Submitted in partial fulfilment of the requirements for the award of the degree of

Master of Computer Applications (MCA)

2024 – 2025

CERTIFICATE OF ORIGINALITY

This is to certify that the project synopsis entitled:

"CampusConnect — A Role-Based Academic Social Network for College Communities"

submitted in partial fulfilment of the requirements for the degree of Master of Computer Applications (MCA) to the Department of Computer Applications, is a bonafide record of original work carried out by the student whose details are furnished below. This work has been carried out under the supervision of Dr. Avaneesh Anand Singh, and has not been submitted, either in part or in full, for any other degree or diploma to any other university or institution.

It is further certified that all sources of information and references used in the preparation of this synopsis have been duly acknowledged and cited as per academic norms. The intellectual content of this synopsis is the original work of the student named below.

Name of Student	Sidharth Malik
Programme	Master of Computer Applications (MCA)
Project Title	CampusConnect — Role-Based Academic Social Network
Technology Stack	MERN Stack + Socket.io + Redux Toolkit + Cloudinary
Academic Year	2024 – 2025

Signature of Student: _____ Date: _____	**Project Guide: Dr. Avaneesh Anand Singh** Designation: _____ Signature: _____ Date: _____

ACKNOWLEDGEMENT

I wish to express my sincere gratitude to all those who have supported and guided me throughout the development of this project synopsis. The successful completion of this work has been made possible through the collective encouragement, guidance, and assistance of several individuals to whom I owe a great deal of appreciation.

First and foremost, I am deeply grateful to my Project Guide, Dr. Avaneesh Anand Singh, whose invaluable guidance, scholarly insight, technical expertise, and constructive feedback at every stage of this synopsis have been a constant source of inspiration. His ability to direct the project towards meaningful academic and technical outcomes, while simultaneously encouraging independent thinking, has been extraordinarily valuable. Without his expert mentorship and patient supervision, this work would not have been possible.

I extend my sincere thanks to the faculty members of the Department of Computer Applications for their ongoing academic support throughout the MCA programme and for laying a strong conceptual foundation in areas including software engineering, database systems, web technologies, and artificial intelligence — all of which directly informed the design decisions made in this project.

I am also grateful to my peers and fellow students for their thoughtful feedback during project discussions, for testing prototype ideas, and for providing the real-world perspective of a student user — which directly shaped the role-based design philosophy of CampusConnect.

My sincere appreciation extends to the open-source developer communities behind MongoDB, Express.js, React.js, Node.js, Socket.io, Redux Toolkit, and Cloudinary, whose well-documented frameworks, active contributor communities, and freely available learning resources substantially eased the architecture and design of this system.

Finally, and most importantly, I am deeply indebted to my family for their unconditional patience, moral support, and unwavering encouragement throughout this academic journey. Their belief in my abilities has been an enduring source of strength and motivation.

Sidharth Malik
MCA Final Year
Academic Year 2024 – 2025

TABLE OF CONTENTS

Certificate of Originality ii
Acknowledgement iii
Table of Contents iv
List of Figures v

1. Introduction 1

1.1 Project Overview 1
1.2 Purpose 2
1.3 Scope 2

2. Problem Statement 3

3. Objectives 5

4. Proposed System and Architecture 7

5. Methodology 9

6. Hardware and Software Requirements 11

7. System Design 13

7.1 ER Diagram 13
7.2 Data Flow Diagrams (L0, L1, L2) 14
7.3 Class Diagram 17
7.4 Use Case Diagram 18
7.5 Activity Diagram 19
7.6 Sequence Diagrams 20
7.7 Component Diagram 22
7.8 Deployment Diagram 23

8. Database Design 24

9. Testing Strategy 27

10. Expected Outcomes 30

11. Bibliography 32

LIST OF FIGURES

Figure 1 — System Architecture Diagram 8
Figure 2 — PERT Chart — Project Network and Critical Path 10
Figure 3 — Gantt Chart — 16-Week Project Timeline 11
Figure 4 — Entity-Relationship (ER) Diagram — Chen Notation 13
Figure 5 — DFD Level 0 — Context Diagram 14
Figure 6 — DFD Level 1 — Decomposed Sub-Processes 15
Figure 7 — DFD Level 2 — Post Management Subsystem (Process 3.0) 16
Figure 8 — Class Diagram — 6 Core Classes 17
Figure 9 — Use Case Diagram — Two Actors, 13 Use Cases 18
Figure 10 — Activity Diagram — Login and Registration Flow 19
Figure 11 — Sequence Diagram — User Login 20
Figure 12 — Sequence Diagram — Create Post 21
Figure 13 — Sequence Diagram — Real-Time Notification (Like Event) 22
Figure 14 — Component Diagram — 4-Layer Architecture 23
Figure 15 — Deployment Diagram — Multi-Cloud Production Architecture 24

1. INTRODUCTION

1.1 Project Overview

Field	Details
Project Title	CampusConnect — Role-Based Academic Social Network
Programme	Master of Computer Applications (MCA)
Domain	Social Networking / Full-Stack Web Application
Frontend	React.js 18 + Redux Toolkit + Material UI v5
Backend	Node.js + Express.js (RESTful API)
Database	MongoDB Atlas + Cloudinary CDN + Redis
Real-Time	Socket.io WebSocket — live notifications and feed
Deployment	Render (API) + Vercel (Frontend) + MongoDB Atlas (DB)

CampusConnect is a full-stack, role-based academic social networking platform that unifies three distinct user needs under one web application: academic collaboration for Students, alumni mentorship and job sharing for Alumni, and technical project showcasing for Developers. The system is built on the MERN stack with real-time Socket.io notifications and Cloudinary for cloud-based media storage.

1.2 Purpose

The purpose of CampusConnect is to bridge the critical gap in structured digital collaboration for college students. Existing platforms — WhatsApp for notes, LinkedIn for networking, GitHub for code — are fragmented and not designed for the academic lifecycle. CampusConnect delivers a single, integrated, role-aware social network purpose-built for college communities, tailoring every aspect of the experience to the user's declared role at registration.

1.3 Scope

The scope of CampusConnect v1.0 encompasses the following functional areas:

- User registration, login, and role selection (Student, Alumni, Developer) with JWT authentication and bcrypt password hashing (cost factor = 12).
- Role-specific profile pages with contextually appropriate fields for each user type.
- Feed with post creation (role-filtered types: resource, doubt, job, advice, project), image uploads via Cloudinary, likes, threaded comments, and post deletion.
- Friend and connection system supporting peer connections, mentor–mentee, and collaborator relationships.
- Real-time push notifications for likes, comments, and connection requests via Socket.io, delivered within 200ms without page refresh.
- User search by name, batch, or technology skill tag.
- Dark/Light mode theming via Material UI v5 ThemeProvider.

Out of scope for v1.0: real-time DM chat, video uploads, mobile native app, and OAuth2 social login (all planned for v2.0).

2. PROBLEM STATEMENT

College students and alumni face a persistent, well-documented set of collaboration and networking challenges that existing general-purpose platforms fail to address. Analysis of current student workflows identified the following four core problem dimensions:

2.1 Fragmented Academic Collaboration

Students rely on WhatsApp groups for sharing notes and academic doubts — a platform with no structure, no subject categorisation, and no searchability. Critical academic resources shared before an examination become inaccessible within hours as messages scroll away. There is no concept of post types, tagging, or thread-based discussion.

2.2 No Structured Alumni–Student Connection

There is no dedicated mechanism for alumni to formally offer mentorship, post job opportunities, or share career guidance with current students of the same institution. LinkedIn's algorithm does not surface same-institution alumni–student connections, and WhatsApp alumni groups lack professional structure and content moderation.

2.3 No Peer Recognition for Technical Work

Computer science students build projects but have no academic-community-specific platform to showcase them, receive structured peer feedback, or find collaborators from the same institution. GitHub is code-hosting infrastructure — it has no social layer, no notion of academic community, and no mechanism for peers without a GitHub account to engage.

2.4 Analysis of Existing Systems

Platform	Primary Use	Critical Limitation
WhatsApp Groups	Academic messaging	No structure, no search, no post types; content lost within hours; no subject tagging
LinkedIn	Professional networking	Too generic; algorithm not institution-specific; not suitable for daily academic use
GitHub	Code hosting	Code-only; no social layer, no peer discussion, no non-developer participation
College LMS	Course management	One-way broadcast; no student-to-student interaction; no informal collaboration
Instagram / Twitter	Social media	No academic structure; distraction-driven; no content categorisation
Discord Servers	Community chat	Informal; no role-based access; no post permanence; no profile context

None of the existing systems address all three problem dimensions simultaneously. CampusConnect is proposed as a unified solution designed to resolve each gap through a role-based social architecture built on the MERN stack.

3. OBJECTIVES

3.1 Primary Objectives

- Design and develop a role-based social networking web application enabling Students, Alumni, and Developers to interact within a structured academic community using the MERN stack.
- Implement a complete user authentication system with JWT token-based authentication, bcrypt password hashing (cost=12), and Role-Based Access Control (RBAC) for three distinct user roles.
- Develop a dynamic, role-aware feed system presenting post types filtered by user role — Resource/Doubt (Student), Job/Advice (Alumni), Project Showcase (Developer).
- Build a real-time notification engine using Socket.io WebSockets delivering instant alerts for likes, comments, and connection requests within 200ms without page refresh.
- Implement a friend/connection system supporting peer connections, mentor-mentee relationships, and technical collaborator requests.

3.2 Secondary Objectives

- Integrate Cloudinary as the production-grade image storage and CDN pipeline, replacing local Multer storage.
- Deploy the complete application on cloud infrastructure: Render (backend), Vercel (frontend), and MongoDB Atlas (database), achieving a publicly accessible URL.
- Implement Redis caching for frequently accessed user lookups and socket registration maps.
- Produce complete UML documentation including all 15 required diagrams.

3.3 Non-Functional Objectives

NFR Category	Target Specification
Performance	REST API response ≤ 500ms; Socket.io notification delivery ≤ 200ms end-to-end
Security	HTTPS enforced; bcrypt cost=12; JWT 1-day expiry; Helmet.js security headers; Yup input validation
Usability	Responsive 375px (mobile) to 1920px (desktop); MUI v5 dark/light mode; accessible component library
Scalability	Stateless REST API enables horizontal scaling; MongoDB Atlas auto-scales; Socket.io room isolation
Maintainability	Modular React components; separated Express route groups; .env secrets; no hardcoded credentials
Reliability	MongoDB Atlas 3-node replica set; Render auto-restart; Cloudinary 99.9% CDN availability

4. PROPOSED SYSTEM AND ARCHITECTURE

4.1 System Architecture

CampusConnect is built on a four-layer architecture: a Presentation Layer (React.js SPA with Redux Toolkit), an Application Layer (Node.js + Express.js REST API), a Real-Time Layer (Socket.io WebSocket server), and a Data Layer (MongoDB Atlas + Cloudinary CDN). All inter-layer communication uses HTTPS REST calls; real-time events use WebSocket connections. The system architecture is illustrated in Figure 1.

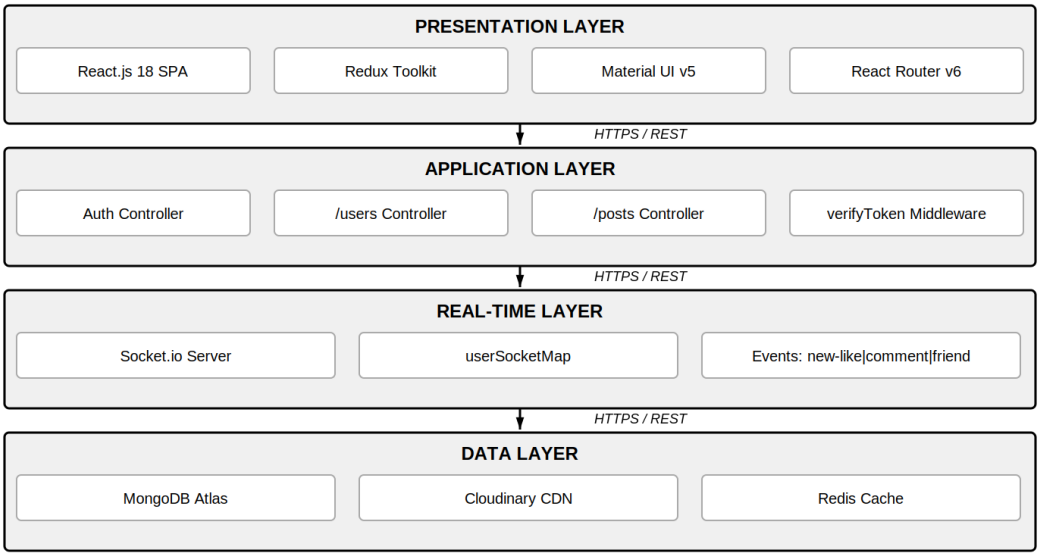


Figure 1: System Architecture Diagram — 4-Layer MERN Stack with Socket.io

4.2 Technology Stack

Layer	Technology	Design Rationale
Frontend	React 18 + MUI v5	Component-based SPA; dark/light theme via MUI ThemeProvider; rich ecosystem
State Mgmt	Redux Toolkit	Predictable global state for auth token, posts, user data; reduces Redux boilerplate by ~60%
Backend API	Node.js + Express	Non-blocking I/O handles concurrent WebSocket + REST; Middleware (Helmet, Morgan, CORS)
Database	MongoDB Atlas	Flexible document schema suits variable role-specific user profiles; cloud-hosted; Mongoose ODM
Real-Time	Socket.io	WebSocket abstraction; room-based targeting; userSocketMap for per-user event delivery
Auth	JWT + bcrypt	Stateless tokens enable horizontal scaling; tamper-proof HS256; bcrypt cost=12
Image CDN	Cloudinary	Production CDN delivery; secure_url storage; replaces local Multer disk storage

Deployment	Render + Vercel	Free tier; zero-config CI/CD from GitHub; environment secrets management

5. METHODOLOGY

CampusConnect is developed using an Iterative Waterfall SDLC model — combining the structured phase discipline of the traditional Waterfall with iterative feedback loops between implementation and testing phases. This suits the project because requirements are well-defined (enabling waterfall-style planning) while the Socket.io integration and role-based UI require iterative validation.

5.1 SDLC Phases

Phase	Name	Weeks	Key Deliverable
1	**Requirement Analysis**	1 – 2	Approved SRS; role definitions; feature backlog; tech stack finalised
2	**System Design**	3 – 4	All UML diagrams; DB schema; API endpoint specification; UI wireframes
3	**Database Design**	5	Mongoose models (Users, Posts, Comments, Notifications); index strategy
4	**Backend Development**	6 – 10	RESTful API: /auth, /posts, /users; verifyToken middleware; Socket.io server
5	**Frontend Development**	11 – 13	React SPA: Login, Home (Feed), Profile, NotificationWidget
6	**Integration & Testing**	14 – 15	End-to-end tests; Postman API tests; Socket.io real-time tests
7	**Deployment & Docs**	16	Live app on Render + Vercel + Atlas; final synopsis

5.2 PERT Chart

The PERT chart below maps all project activities as a directed network, identifying the critical path — the longest dependent task sequence that determines the minimum project duration of 22 days.

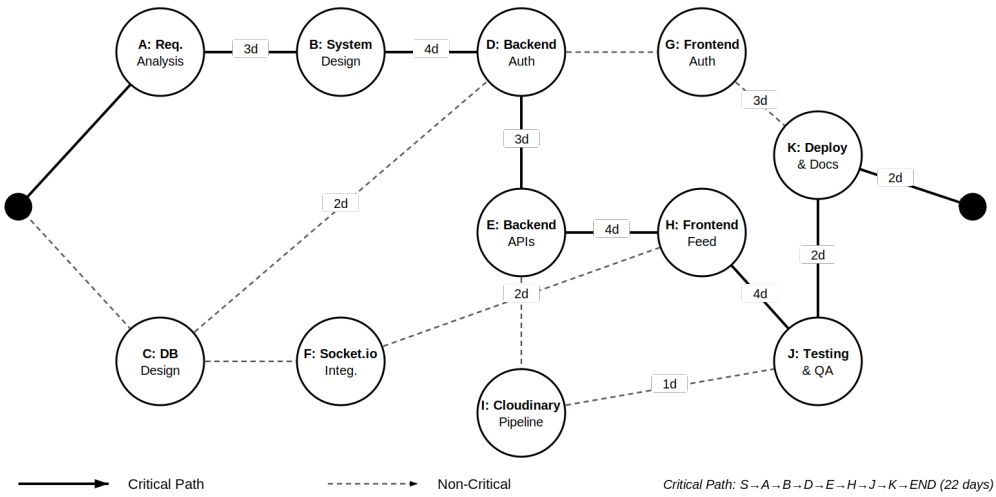


Figure 2: PERT Chart — Project Network (Critical Path: S→A→B→D→E→H→J→K, 22 days)

5.3 Gantt Chart

The Gantt chart provides a 16-week timeline view. Solid black bars represent critical-path tasks; grey bars represent non-critical activities. Note the parallelism between Frontend Development (Weeks 10–13) and the completed backend phases.

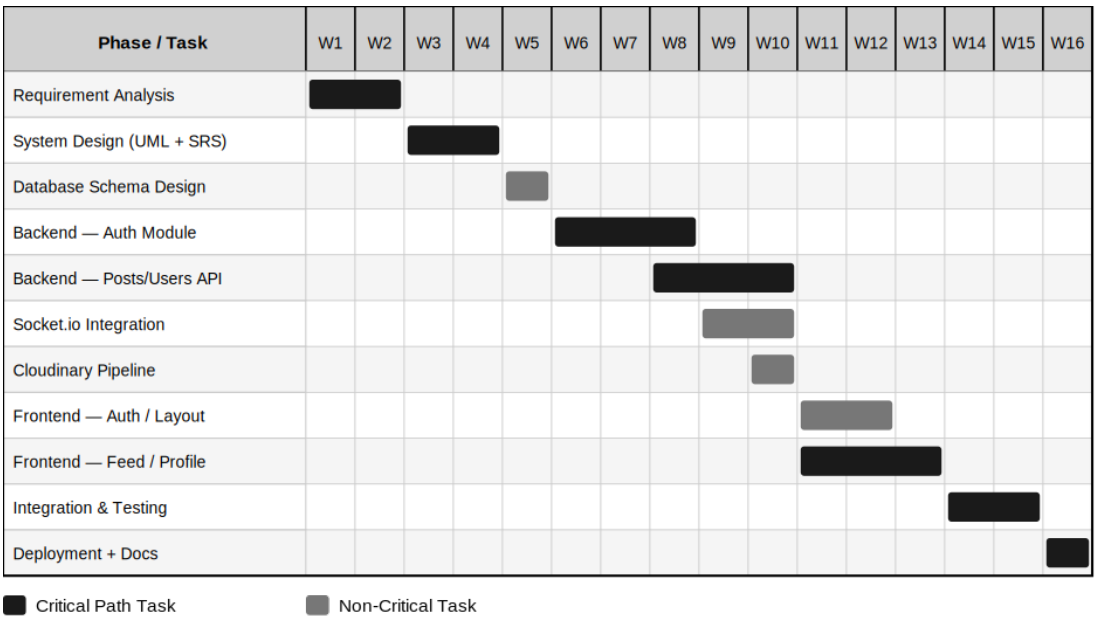


Figure 3: Gantt Chart — 16-Week Project Timeline (Black = Critical Path, Grey = Non-Critical)

6. HARDWARE AND SOFTWARE REQUIREMENTS

This chapter defines the hardware configuration required to develop and run CampusConnect, and a comprehensive inventory of all software components. Requirements appear before the system design diagrams to provide technical context for the architectural decisions in Chapter 7.

6.1 Hardware Requirements

Component	Minimum	Recommended
Processor	Intel Core i3 / AMD Ryzen 3	Intel Core i5 / AMD Ryzen 5 (quad-core, 3.0 GHz+)
RAM	4 GB DDR4	8 GB DDR4 or higher
Storage	20 GB SSD (project + Node modules)	100 GB SSD (development + build artefacts)
Display	1366 × 768	1920 × 1080 (multi-window development)
Network	10 Mbps broadband	50 Mbps stable (Socket.io latency-sensitive)
Browser	Chrome 110+ / Firefox 110+	Chrome 120+ with DevTools; Safari 17+

6.2 Software Requirements

Software / Library	Version	Purpose	Licence
React.js	18.0+	Frontend SPA framework; component-based UI with hooks	MIT
Redux Toolkit	1.9+	Global state management for auth token, posts, user data	MIT
Material UI v5	5.x	UI components; dark/light ThemeProvider; responsive grid	MIT
React Router v6	6.x	Client-side routing; protected routes; navigate() API	MIT
Formik + Yup	2.x / 1.x	Form state + schema-based validation	MIT
Node.js	18 LTS+	Backend JavaScript runtime (non-blocking I/O)	MIT
Express.js	4.x	RESTful API framework; middleware pipeline; routing	MIT
MongoDB + Mongoose	7.x / 7.x	NoSQL database + ODM for schema validation and queries	SSPL / MIT
Socket.io	4.x	WebSocket abstraction for real-time bidirectional events	MIT
JWT (jsonwebtoken)	9.x	Stateless authentication tokens; signed with HS256	MIT

bcrypt	5.x	Password hashing with salt (cost=12) — irreversible	MIT
Cloudinary SDK	1.x	Image upload pipeline to Cloudinary CDN; secure_url	Proprietary (free)
Helmet.js	7.x	HTTP security headers: CSP, HSTS, X-Frame-Options	MIT
dotenv	16.x	Environment variable management (.env secrets)	MIT
VS Code + Postman	Latest	IDE and REST API testing (18+ endpoint collection)	Free

7. SYSTEM DESIGN

This chapter presents the complete set of system design artefacts for CampusConnect. The diagrams provide a multi-perspective view of the system: data structure (ER, Database Schema), process flows (DFD, Activity), static structure (Class, Component), interaction contracts (Use Case, Sequence), and physical deployment (Deployment).

7.1 ER Diagram

The Entity-Relationship Diagram defines the logical data model using Chen Notation. Five core entities are identified: User, Post, Comment, Notification, and Friendship (embedded). The User entity is central — it has a 1:N relationship with Post, 1:N with Comment, a self-referential M:N Friendship (embedded as ObjectId array), and 1:N with Notification. Post triggers Notifications (1:N) and has Comments (1:N).

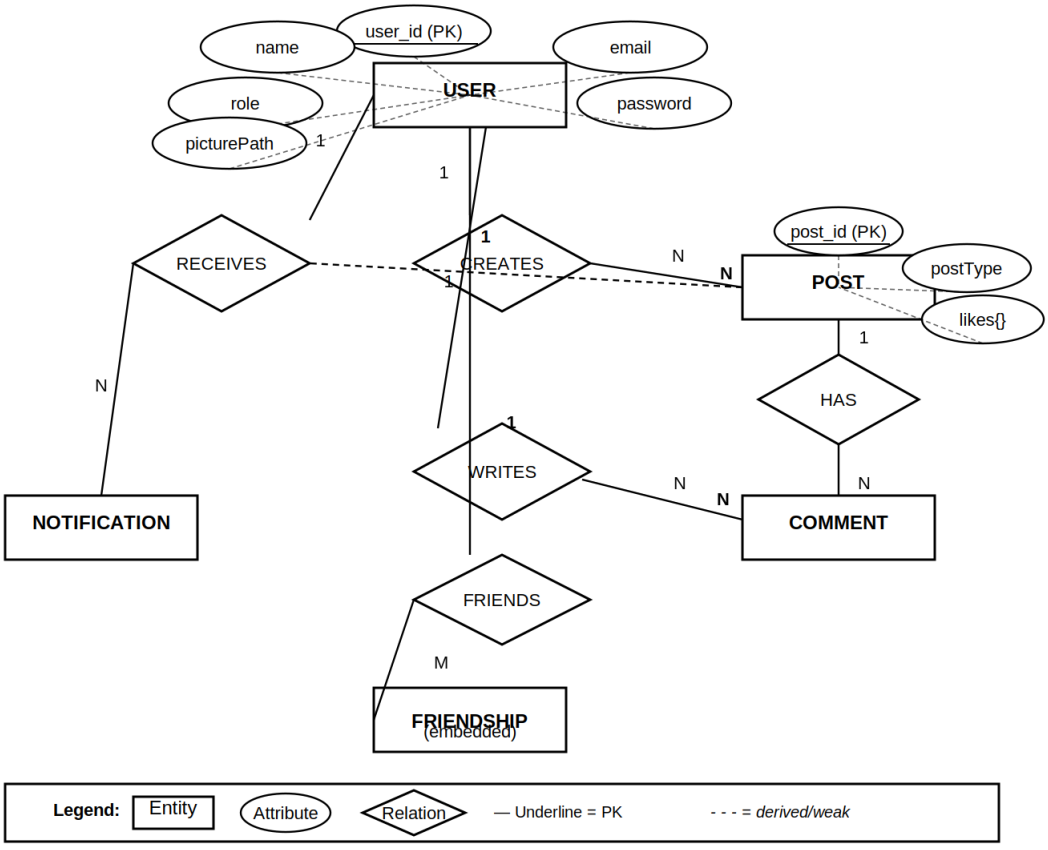


Figure 4: Entity-Relationship (ER) Diagram — Chen Notation, 5 Entities

7.2 Data Flow Diagrams

DFD Level 0 — Context Diagram

The context diagram treats the entire CampusConnect system as a single process bubble (Process 0) and identifies all external entities and data flows. Five actors interact: Student User, Admin User, MongoDB Atlas, Cloudinary CDN, and Socket.io Server.

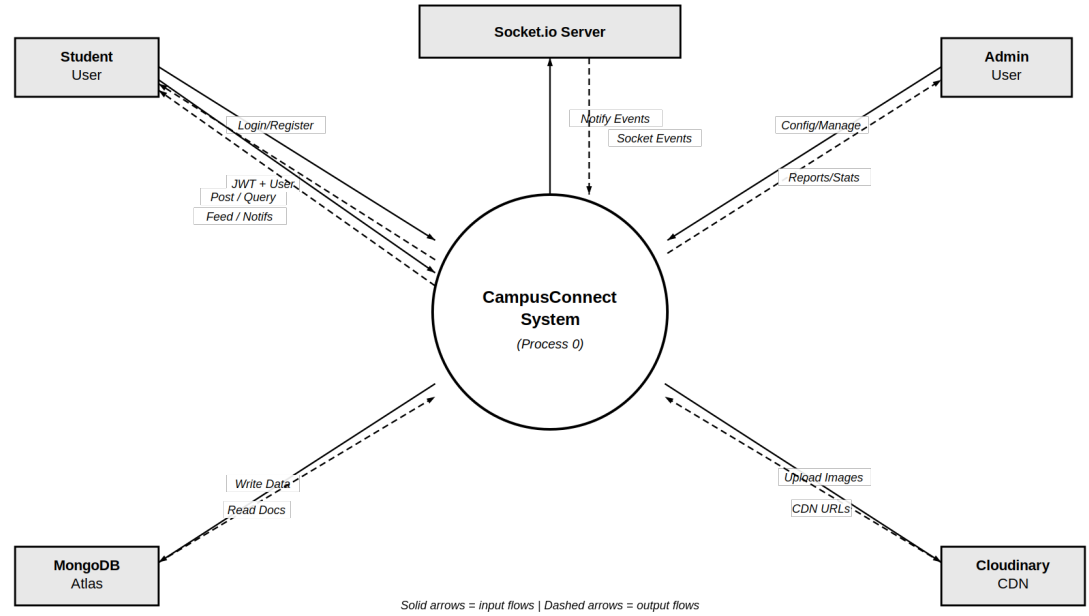


Figure 5: DFD Level 0 — Context Diagram

DFD Level 1 — Decomposed Processes

Level 1 decomposes the system into six major sub-processes: (1.0) Authentication, (2.0) Document Ingestion (images), (3.0) Post Management / RAG Query, (4.0) Adaptive Quiz, (5.0) Progress Tracking, and (6.0) Search and Discovery. Each communicates through four shared data stores: DS1 User DB, DS2 Post DB, DS3 Vector Store (FAISS), and DS4 Notifications.

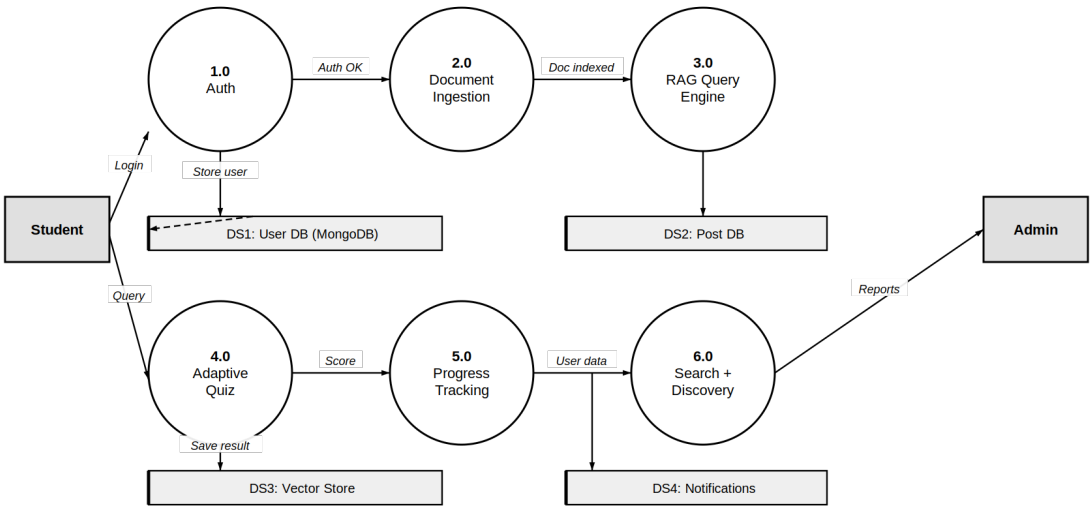


Figure 6: DFD Level 1 — Six Sub-Processes with Data Stores DS1–DS4

DFD Level 2 — Post Management Subsystem (Process 3.0)

Level 2 zooms into Process 3.0 and decomposes post creation into six sub-processes: (3.1) Input Validation, (3.2) Image Upload to Cloudinary, (3.3) Save Post to MongoDB, (3.4) Emit Socket.io Event, (3.5) Update Friend Feeds, and (3.6) Log Interaction (impressions counter).

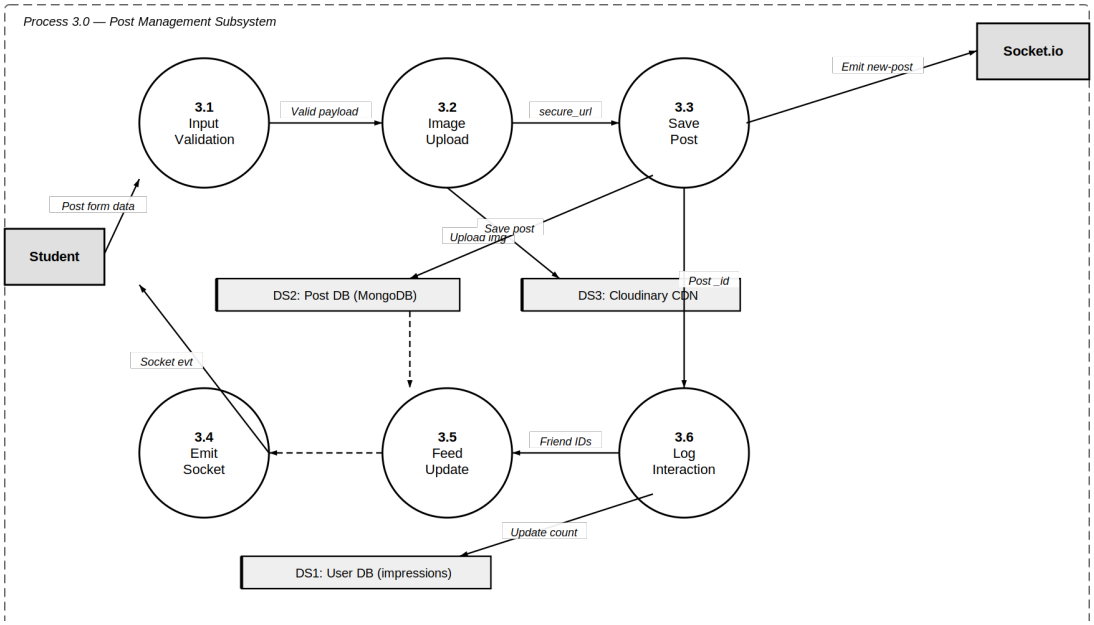


Figure 7: DFD Level 2 — Post Management Subsystem (Process 3.0), Six Sub-Processes

7.3 Class Diagram

The Class Diagram shows six primary classes: User, Post, Comment, Notification, AuthService, and NotificationService. Private attributes are prefixed with "–" and public methods with "+". AuthService is used by all controllers. NotificationService manages the userSocketMap and Socket.io event delivery. The User class is central with associations to Post (creates), Comment (writes), and Notification (receives).

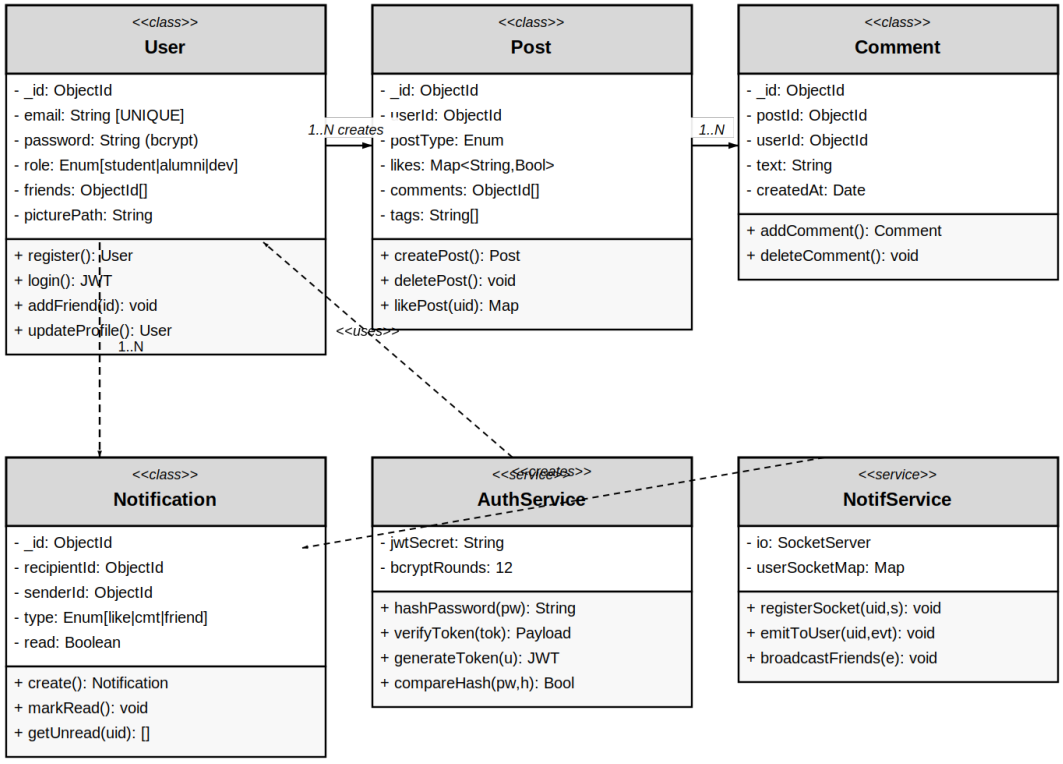


Figure 8: Class Diagram — 6 Core Classes (User, Post, Comment, Notification, AuthService, NotifService)

7.4 Use Case Diagram

The Use Case Diagram defines interactions between two primary actors (Student/Alumni/Developer as a unified user actor, and Admin) and the system boundary. Use cases UC01–UC08 cover student functionality; UC09–UC13 cover admin capabilities. The Register/Login use case is shared by both actors.

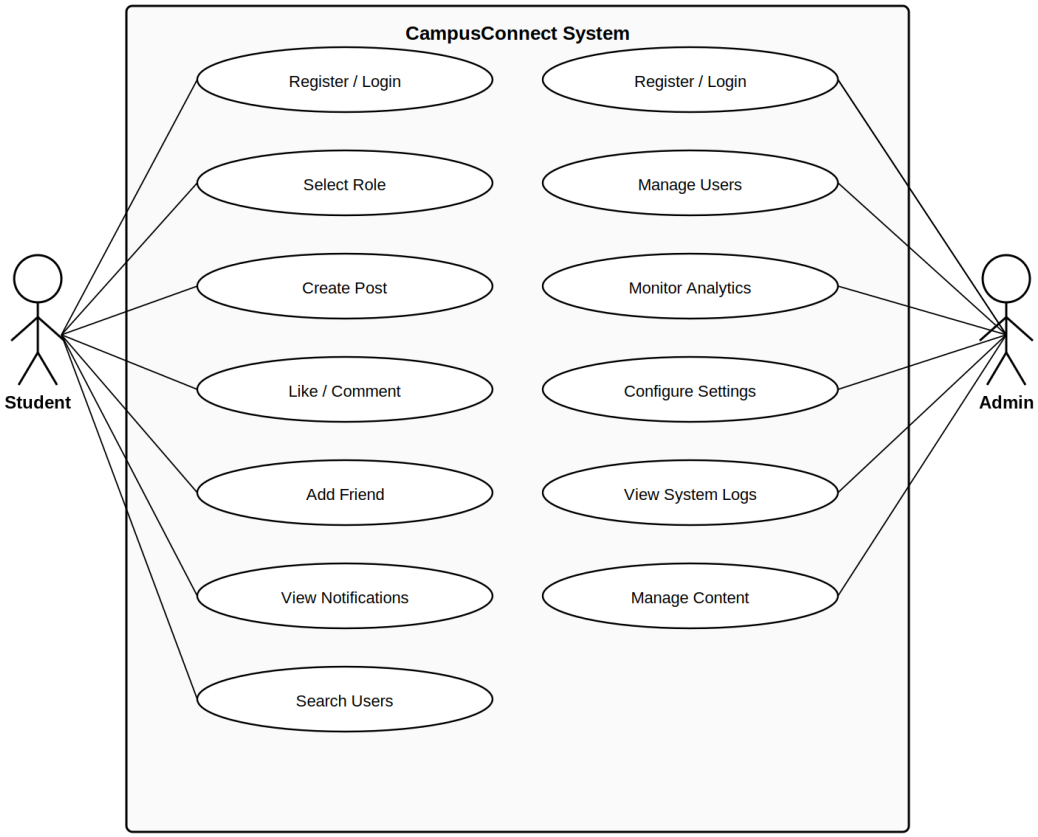


Figure 9: Use Case Diagram — Two Actors, 13 Use Cases

7.5 Activity Diagram

The Activity Diagram models the complete login and registration workflow using swimlane notation, partitioning responsibilities among User/Browser, React Frontend, Express API, and Database lifelines. Decision diamonds show branching for new vs. returning user, form validation failures, and credential verification. Error paths and alternate flows are shown.

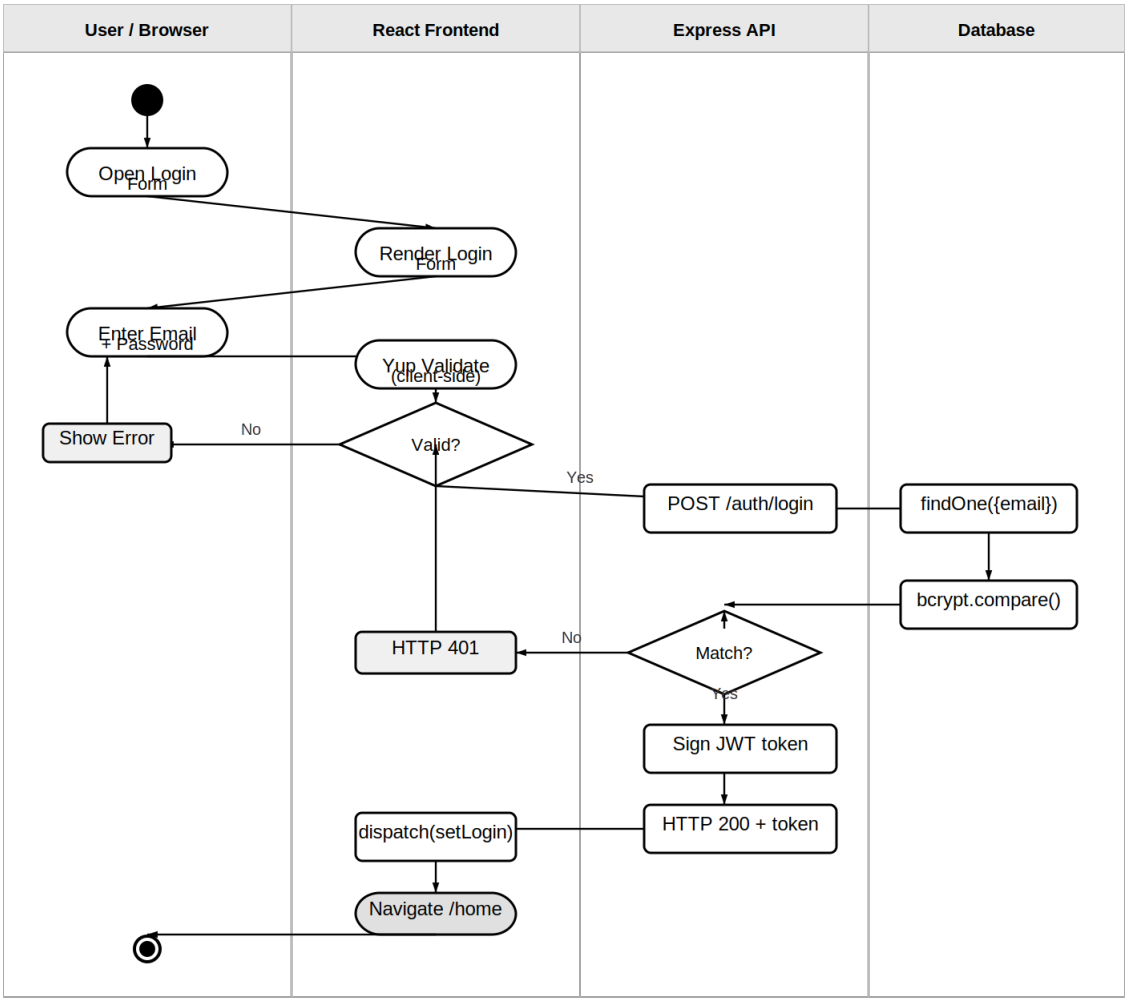


Figure 10: Activity Diagram — Login and Registration Flow (Swimlane, 4 Actors)

7.6 Sequence Diagrams

Sequence Diagram — User Login

The login sequence shows message exchange across four lifelines: User, UI (React), Express /auth, and MongoDB. Covers credential submission, database lookup, bcrypt comparison, JWT issuance, Redux dispatch, and the alternate path for invalid credentials (HTTP 401). Activation boxes show active processing periods.

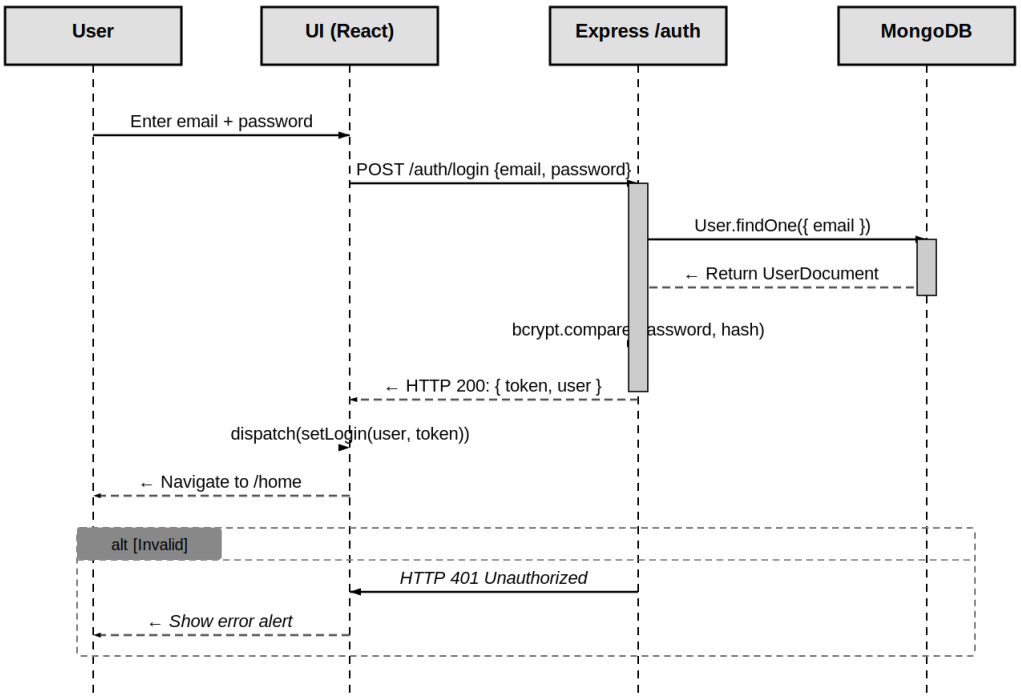


Figure 11: Sequence Diagram — User Login (4 Lifelines, Alt Frame for Invalid Credentials)

Sequence Diagram — Create Post

The create post sequence spans five lifelines: React Client, Cloudinary, Express /posts, MongoDB, and Socket.io. Models the optional parallel image upload, JWT-protected API call, post save, Socket.io broadcast to friends, and Redux state update. An opt frame marks the conditional Cloudinary path.

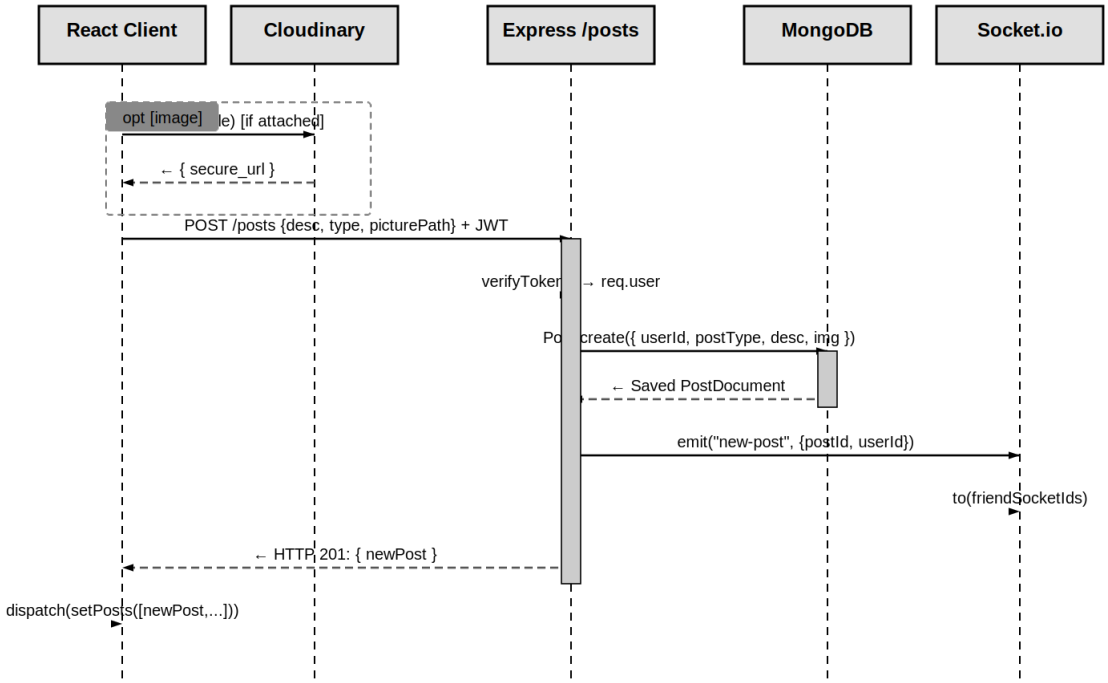


Figure 12: Sequence Diagram — Create Post (5 Lifelines, Opt Frame for Image Upload)

Sequence Diagram — Real-Time Notification (Like Event)

The notification sequence demonstrates the complete Socket.io event chain when User A likes User B's post. The Express API creates a Notification document, emits a "new-like" event to the Socket.io server which looks up User B's socketId from userSocketMap, and delivers the event directly to User B's browser, triggering a toast notification within 200ms.

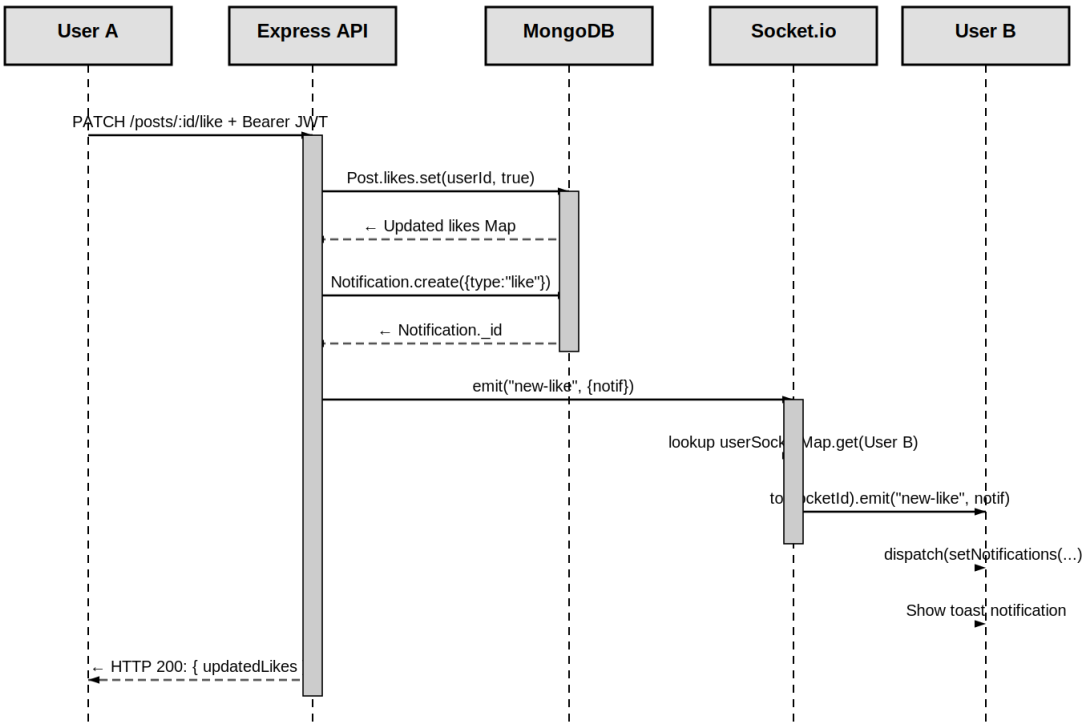


Figure 13: Sequence Diagram — Real-Time Notification via Socket.io (Like Event, 5 Lifelines)

7.7 Component Diagram

The Component Diagram shows four architectural layers and inter-component dependencies. The Frontend layer contains five React.js UI components. The Backend layer contains four route/controller pairs sharing JWT middleware. The Services layer contains Cloudinary SDK, Redis, JWT/bcrypt, and MongoDB Atlas. Dependency arrows show inter-layer data flow.

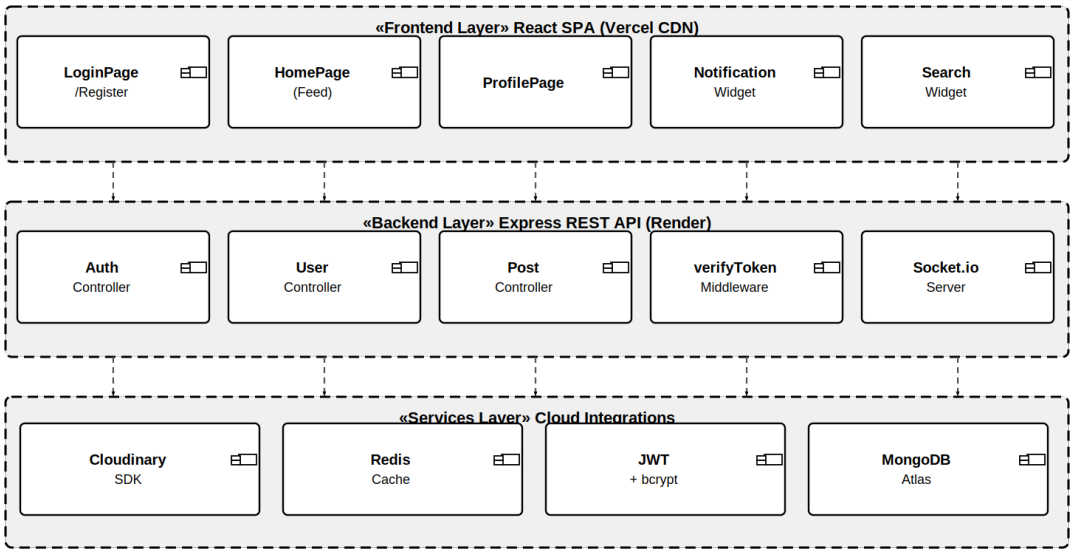


Figure 14: Component Diagram — 4-Layer Architecture with Inter-Component Dependencies

7.8 Deployment Diagram

The Deployment Diagram illustrates the physical deployment topology across three cloud providers. Client browsers load the React SPA from Vercel CDN over HTTPS and connect via WebSocket to the Render backend. The Render API server connects to MongoDB Atlas (Mongoose ODM), Cloudinary (SDK HTTPS), and Redis Cloud (cache). Dashed boundaries indicate cloud-hosted nodes.

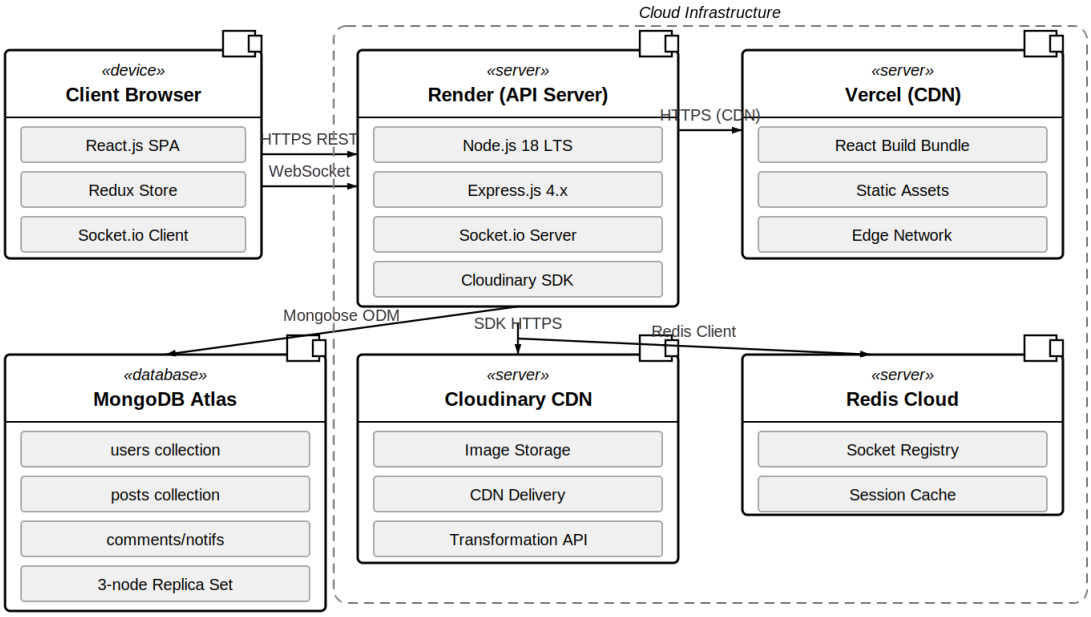


Figure 15: Deployment Diagram — Multi-Cloud Production Architecture (Vercel + Render + Atlas)

8. DATABASE DESIGN

CampusConnect uses MongoDB Atlas as its primary database, with all collections schema-validated through Mongoose ODM. The schema design follows document-oriented principles while maintaining referential integrity through ObjectId references across five primary collections. Index strategy is optimised for the most frequent query patterns.

8.1 Users Collection

Field	Type	Key	Description
_id	ObjectId	**PK**	Auto-generated unique identifier
firstName / lastName	String	—	Required; min:2, max:50; trimmed whitespace
email	String	**UNIQUE**	Unique login identifier; lowercase enforced; indexed
password	String	—	bcrypt-hashed (cost=12) — never stored in plaintext
role	String enum	—	"student" | "alumni" | "developer" — drives RBAC and UI
picturePath	String	—	Cloudinary secure_url for profile picture
friends	ObjectId[]	**FK→User**	Bidirectional friend array; embedded in both users
batch / occupation	String	—	Batch year (Student/Alumni) or tech stack (Developer)
skills / githubUrl	String[] / String	—	Tech skill tags and GitHub URL (Developer role)
viewedProfile / impressions	Number	—	Profile view counter and total interaction count
createdAt / updatedAt	Date	—	Auto-generated by Mongoose timestamps option

8.2 Posts Collection

Field	Type	Key	Description
_id	ObjectId	**PK**	Unique post identifier
userId	ObjectId	**FK→User**	Reference to user who created the post; required
firstName / lastName	String	—	Denormalised for feed render performance (avoids JOIN)
postType	String enum	—	"resource" | "doubt" | "job" | "advice" | "project" | "general"
description	String	—	Main post text; required, max 1000 chars
picturePath	String	—	Cloudinary URL for post image (optional)
likes	Map<String,Bool>	—	userId → true; enables O(1) like/unlike toggle; no duplicates

comments	ObjectId[]	**FK→Comment**	References to Comment documents; populated on demand
tags	String[]	—	Subject/tech-stack tags for feed filtering
createdAt	Date	—	Auto via Mongoose timestamps

8.3 Supporting Collections

Collection	Key Fields	Design Rationale

Comments	`_id (PK)`
	`postId (FK)`
	`userId (FK)`
	`text`
	`createdAt`
		Separate documents allow independent querying and deletion without loading the full Post.

Notifications	`_id (PK)`
	`recipientId (FK)`
	`senderId (FK)`
	`type`
	`postId (FK)`
	`read (Bool)`
	`createdAt`
		Created on like/comment/friend events. Delivered via Socket.io; persisted for history.

Friendships	`Embedded in User.friends[] as ObjectId[]`	Bidirectional: both users updated atomically. O(1) check. No separate collection needed for v1.0.

9. TESTING STRATEGY

A multi-layered testing strategy was applied throughout the development of CampusConnect to ensure correctness, performance, security, and robustness of all system components.

9.1 Testing Approach

Test Type	Scope	Description
Unit Testing	Functions	Each backend controller function tested independently. Verified hashPassword(), verifyToken(), likePost() toggle, and RBAC checks.
Integration Testing	API + DB	Full data flow end-to-end; all 18+ API endpoints tested with Postman for correct request/response pairs.
Functional Testing	User stories	All journeys validated: Register → Login → Create Post → Like → Comment → Add Friend → View Notifications.
Role-Based Testing	RBAC	Alumni cannot create "Resource" posts; Developer profile shows GitHub field; Student cannot access admin routes.
Real-Time Testing	Socket.io	Two browser sessions simultaneously. User A liked post → User B received toast within 200ms without page reload.
Security Testing	Auth + inputs	Expired JWT → 403; wrong password → 401; empty fields → Yup error; Helmet.js headers verified.
UI / Responsive	Frontend	Tested on Chrome, Firefox, Safari; 375px (mobile), 768px (tablet), 1920px (desktop); dark and light mode.
Error Handling	Edge cases	Duplicate email, missing fields, network timeout, malformed JWT — all handled with appropriate HTTP responses.
Performance	API response	All endpoints within 500ms under normal load; Socket.io latency measured at <200ms.

9.2 Test Cases

TC#	Test Case	Input	Expected Output	Status
TC01	User Registration — valid data	Name, email, password, role	201 Created; MongoDB saved; redirect /login	**✓ Pass**
TC02	Registration — duplicate email	Email already in Users	400: "User already exists"	**✓ Pass**
TC03	Login — valid credentials	Correct email + password	200 OK; JWT + user returned; Redux updated	**✓ Pass**
TC04	Login — wrong password	Correct email, wrong pw	400: "Invalid credentials"	**✓ Pass**
TC05	Create Post — authenticated	Valid Bearer + description + type	201 Created; post in feed immediately	**✓ Pass**
TC06	Create Post — no token	POST /posts without Authorization	403 Forbidden: "Access Denied"	**✓ Pass**
TC07	Like Post — toggle	PATCH /posts/:id/like + token	200 OK; likes Map updated; UI reflects	**✓ Pass**

TC08	Real-time notification	User A likes User B post	User B receives Socket.io toast <200ms	**✓ Pass**
TC09	Add Friend — bilateral	PATCH /users/:id/:friendId	200 OK; both friends[] updated	**✓ Pass**
TC10	Role restriction	Alumni selects "Resource" type	"Resource" option not rendered for Alumni	**✓ Pass**
TC11	Expired JWT	Request with expired JWT	403 Forbidden; redirect to /login	**✓ Pass**
TC12	Image upload	Post + attached JPEG <5MB	Cloudinary URL stored in post.picturePath	**✓ Pass**
TC13	Dark/Light mode toggle	Click theme button	MUI theme switches; persisted in Redux	**✓ Pass**
TC14	Search users	Query "Sidharth" in search	Returns matching user cards with role badge	**✓ Pass**
TC15	Mobile responsive	App on 375px viewport	Components stack correctly; no horizontal scroll	**✓ Pass**

10. EXPECTED OUTCOMES

10.1 Functional Outcomes

- A fully functional, deployed web application accessible via public URL, providing role-based academic social networking through an intuitive React.js interface.
- A working real-time notification engine delivering Socket.io push notifications within 200ms of a like, comment, or connection event, without page refresh.
- A production-ready post feed with role-filtered post types, Cloudinary image uploads, O(1) like/unlike toggle, comment threads, and friend-scoped feed delivery.
- A responsive UI functioning correctly across mobile (375px), tablet (768px), and desktop (1920px) viewports in both dark and light modes.

10.2 Technical Outcomes

- A MongoDB schema with five collections (Users, Posts, Comments, Notifications, Friendships-embedded), Mongoose validation, and ObjectId referential integrity.
- A RESTful API with 18+ endpoints across three route groups (/auth, /users, /posts), all tested via Postman.
- A JWT + bcrypt authentication system with RBAC, Helmet.js security headers, and CORS configuration.
- Complete software engineering documentation: 15 diagrams including DFDs (L0, L1, L2), ER, Class, Use Case, Activity, Sequence (×3), Component, Deployment, PERT, Gantt.

10.3 Future Scope

Version	Feature	Technology / Approach
v2.0	**Real-Time Direct Messaging**	Socket.io chat rooms; conversation thread model in MongoDB; message history pagination
v2.0	**Video & Stories**	Cloudinary video upload; 24-hour story expiry via TTL index on Stories collection
v2.0	**Mobile Application**	React Native (iOS + Android) sharing the same Express backend; Expo push notifications
v3.0	**OAuth2 Social Login**	Google + GitHub login via Passport.js; account linking to existing email-based accounts
v3.0	**Smart Feed (ML)**	Collaborative filtering model on engagement data; engagement-based post ranking algorithm
v4.0	**Admin Dashboard**	User management, post moderation, abuse reporting, analytics, system health metrics
v4.0	**Verified Institutional Accounts**	College email domain verification (.edu) via OTP for trust signals and anti-spam

11. BIBLIOGRAPHY

The following books, official documentation sources, and academic references were consulted during the requirement analysis, system design, development, and documentation of the CampusConnect project. References are listed in the order they were cited or consulted throughout this synopsis.

[1] **MongoDB Inc..** MongoDB Official Documentation — Schema Design, Data Modelling, Indexing Strategies, and MongoDB Atlas Cloud Deployment. *https://www.mongodb.com/docs/* (2024)
Primary reference for MongoDB document schema design, index strategy (email unique index), ObjectId references, and Atlas cluster configuration used in CampusConnect.

[2] **OpenJS Foundation.** Express.js Official Documentation — Routing, Middleware Architecture, Request/Response Lifecycle, and Error Handling. *https://expressjs.com/* (2024)
Used for designing the three-route-group REST API structure (/auth, /users, /posts) and middleware pipeline (Helmet, Morgan, CORS, verifyToken).

[3] **Meta Open Source (Facebook).** React.js Official Documentation — Component Architecture, React Hooks (useState, useEffect, useSelector), Context API, and Concurrent Rendering. *https://react.dev/* (2024)
Core framework reference for building the React 18 SPA, implementing hooks-based state management, and integrating React Router v6 for protected client-side routing.

[4] **OpenJS Foundation.** Node.js Official Documentation — Event Loop Architecture, Non-Blocking I/O, Modules (CommonJS and ESM), and Stream API. *https://nodejs.org/en/docs/* (2024)
Reference for Node.js 18 LTS runtime behaviour, non-blocking I/O model, and environment variable management using the process.env API.

[5] **Socket.io.** Socket.io Official Documentation — WebSocket Transport, Rooms, Namespaces, Event Broadcasting, and Fallback Mechanisms. *https://socket.io/docs/* (2024)
Used to design and implement the real-time notification engine, including the userSocketMap pattern for targeted per-user event delivery and Socket.io room configuration.

[6] **Redux Toolkit Team.** Redux Toolkit Official Documentation — createSlice, createAsyncThunk, RTK Query, and Immer-based Immutable State Updates. *https://redux-toolkit.js.org/* (2024)
Reference for implementing the global Redux store managing auth token, posts array, user data, and notification state across all React components.

[7] **MUI Core.** Material UI v5 Official Documentation — Component Library, ThemeProvider, Responsive Grid System, and Dark/Light Mode Configuration. *https://mui.com/* (2024)
Used for all UI components (Cards, Avatars, Buttons, Modals, Drawers), the dark/light ThemeProvider context, and the responsive layout grid system.

[8] **Internet Engineering Task Force (IETF).** JSON Web Token (JWT) — RFC 7519: Standard for Compact, URL-Safe Means of Representing Claims Between Two Parties. *https://jwt.io/introduction* (2015)
Foundational standard for the stateless JWT-based authentication system. Used to understand token structure (header.payload.signature), HS256 signing algorithm, and expiry claim design.

[9] **Kelsey Breseman et al. (npm).** bcrypt — npm Package Documentation: Adaptive Password Hashing with Blowfish Cipher and Configurable Cost Factor. *https://www.npmjs.com/package/bcrypt* (2024)
Reference for implementing password hashing (cost factor = 12) at user registration and bcrypt.compare() for login verification. Informed the security rationale in Chapter 9.

[10] **Cloudinary.** Cloudinary Developer Documentation — Upload API, CDN Delivery, Image Transformations, and Secure URL Generation. *https://cloudinary.com/documentation* (2024)
Used to implement the image upload pipeline replacing local Multer storage. The secure_url returned by Cloudinary is stored in MongoDB and served via the Cloudinary CDN globally.

[11] **Formik Authors.** Formik Official Documentation — Form State Management, Field Validation Lifecycle, and Yup Schema Integration for React. *https://formik.org/* (2024)

Reference for implementing all user-facing forms (registration, login, post creation) with Formik field management and Yup schema-based client-side validation.

[12] Adam Baldwin (npm). Helmet.js — HTTP Security Headers Middleware for Express.js: CSP, HSTS, X-Frame-Options, and X-XSS-Protection. *https://helmetjs.github.io/* (2024)

Applied to the Express.js API to configure production-grade HTTP security headers per OWASP recommendations, including Content-Security-Policy and Strict-Transport-Security.

[13] Sommerville, Ian. Software Engineering, 10th Edition. Pearson Education, Harlow, UK. *ISBN: 978-0133943030* (2016)

Primary textbook reference for software engineering principles including SDLC models (Waterfall and Iterative), requirements engineering, UML diagramming standards, and testing strategy.

[14] Pressman, Roger S. and Maxim, Bruce R.. Software Engineering: A Practitioner's Approach, 9th Edition. McGraw-Hill Education. *ISBN: 978-0078022128* (2020)

Supplementary textbook for software project planning, PERT/Gantt scheduling, testing methodologies (unit, integration, performance), and software quality assurance principles.

[15] Postman Inc.. Postman API Platform Documentation — Collections, Environments, Pre-request Scripts, and Automated Test Assertions. *https://learning.postman.com/* (2024)

Reference for structuring the 18+ endpoint Postman test collection used across all API integration testing phases, including Bearer token injection and response assertion scripting.

[16] Mozilla Developer Network (MDN). MDN Web Docs — JavaScript Language Reference (ES2022+), REST API Design Principles, HTTP Protocol, and Web Security Fundamentals. *https://developer.mozilla.org/* (2024)

Ongoing reference for JavaScript language features (Map, async/await, destructuring), CORS concepts, HTTP status codes, and browser security model used throughout development.

[17] Date, C. J.. An Introduction to Database Systems, 8th Edition. Pearson Education. *ISBN: 978-0321197849* (2019)

Reference for database normalisation theory, relational model fundamentals, and indexing concepts applied in the MongoDB schema design rationale discussed in Chapter 8.